OS UNIT 4 ORANGE PROBLEM

NAME: SHAURYA SINGH

SRN: PES1UG24CS437

Screenshots:

1A and 1B:

```
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$
```

2A and 2B:

```
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

3A and 3B:

```
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ make pes
make: 'pes' is up to date.
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ echo "hello" > file1.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ echo "world" > file2.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes add file1.txt file2.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h

shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ cat .pes/index
100755 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776412028 6 file1.txt
100755 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776412030 6 file2.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$
```

4A, 4B and 4C:

```
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ make pes
make: 'pes' is up to date.
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ echo "Hello" > hello.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes add hello.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes commit -m "initial commit"
Committed: 6c2625b4a876... initial commit
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ echo "world" >> hello.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes add hello.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes commit -m "Added world"
Committed: a075f8ff9859... Added world
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ echo "Goodbye" > bye.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes add bye.txt
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes commit -m "Added farewell"
Committed: 59f5efc1271f... Added farewell
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ ./pes log
commit 59f5efc1271fe4273ea23e9572f8066c8d68c9984adff34bb86684077d80f4b1
Author: Shaurya Singh PES1UG24CS437
Date:   1776412421

    Added farewell

commit a075f8ff9859dcbe67476cca2d66c0f500f2279fafa456d60b5ba7cee62bb865
Author: Shaurya Singh PES1UG24CS437
Date:   1776412392

    Added worldAdded farewell

commit 6c2625b4a87686449b3eb4a42f4fca2b1860639286ce775f2305758056165a11
Author: Shaurya Singh PES1UG24CS437
Date:   1776412359

    initial committialQ

shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$
```

Final Screenshot:

```
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 376c40344e8f... Initial commit

Log after first commit:
commit 376c40344e8f5947ccf3f17572131ae5b073beffb1a55dcab3b26769afe39273
Author: Shaurya Singh PES1UG24CS437
Date:   1776412816

    Initial commit


--- Second Commit ---
Committed: e8a70954219c... Update file.txt

--- Third Commit ---
Committed: ccd4868c8e9c... Add farewell

--- Full History ---
commit ccd4868c8e9c35fdfb845e8697e4fa16f346d15e6c002889cc4ac1a7384bcf78
Author: Shaurya Singh PES1UG24CS437
Date:   1776412816

    Add farewell

commit e8a70954219cb0aeb5e9bca9c331bb48a7593258b176453fc35b1a19798b2429
Author: Shaurya Singh PES1UG24CS437
Date:   1776412816

    Update file.txtfarewell

commit 376c40344e8f5947ccf3f17572131ae5b073beffb1a55dcab3b26769afe39273
Author: Shaurya Singh PES1UG24CS437
Date:   1776412816

    Initial committialQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
ccd4868c8e9c35fdfb845e8697e4fa16f346d15e6c002889cc4ac1a7384bcf78

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/37/6c40344e8f5947ccf3f17572131ae5b073beffb1a55dcab3b26769afe39273
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/cc/d4868c8e9c35fdfb845e8697e4fa16f346d15e6c002889cc4ac1a7384bcf78
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e8/a70954219cb0aeb5e9bca9c331bb48a7593258b176453fc35b1a19798b2429

=== All integration tests completed ===
shaurya@SHAURYA69420:/mnt/c/Users/Shaurya/Desktop/PES1UG24CS437-pes-vcs$
```

ANAYLYSIS:

5.1) Three things need to change in .pes/: HEAD must be updated to ref: refs/heads/<branch>, and the working directory files must be updated to match the target commit's tree. The complexity comes from the working directory update. You must recursively walk the target tree, for each blob entry read the object and overwrite the file, and delete any files tracked in the current index that don't exist in the target tree. This is complex because you must handle subdirectories recursively, preserve untracked files, and do it atomically enough that a crash mid-checkout doesn't leave a corrupted state.

5.2) For each file in the current index, compare its stored mtime and size against the actual filesystem values using stat(). If they differ, the file is locally modified. Then check whether that same path exists in the target branch's tree with a different blob hash. If both conditions are true, locally modified and different between branches, the checkout is refused and a conflict is reported. This is exactly the fast-path check index_status already does, extended to also compare against the target tree.

5.3) In detached HEAD state, HEAD contains a raw commit hash instead of ref: refs/heads/main. New commits are created and chained normally, but no branch file is updated, so the new commits are only reachable by following from HEAD directly. If you switch branches, HEAD is overwritten and those commits become unreachable, i.e., no branch points to them. To recover them, you'd need the hash of the last commit you made (e.g., from terminal scroll-back or pes log while still detached), then run pes checkout -b <new-branch> <hash> to create a branch pointing at it before switching away.

6.1) Use a mark-and-sweep approach:

1. Mark phase**:** Start from every branch ref (all files in .pes/refs/heads/). For each, walk the commit chain following parent pointers. For each commit, add its tree hash to a reachable set, then recursively walk the tree, adding every subtree and blob hash encountered. Use a hash set (e.g., a hash table or sorted array) to track visited hashes and avoid cycles.
2. Sweep phase**:** Walk every file under .pes/objects/. For each, reconstruct its hash from the path (XX + YYY...). If it's not in the reachable set, delete it.

For 100,000 commits and 50 branches: assuming an average of around 20 tree/blob objects per commit, you'd visit roughly 2,000,000 objects in the mark phase. With a hash set, each lookup is O(1), making the full GC O(n) in the number of objects.

6.2) The race:

1. A commit operation calls object_write for a new blob, the blob is written to disk but the commit object referencing it hasn't been written yet.
2. GC runs its mark phase at this exact moment. The blob isn't reachable from any branch (the commit doesn't exist yet), so it's not marked.
3. GC's sweep phase deletes the blob.
4. The commit operation now writes the commit object referencing the deleted blob. The repository is corrupt.

Git avoids this by using a grace period: objects newer than 2 weeks are never deleted by GC regardless of reachability. This means a new object written during a concurrent commit is safe because it will be younger than the grace period threshold. Additionally, Git writes a FETCH_HEAD / lock file during operations to signal active writers.